

Border Defence AI - Project Understanding Notebook

Integrated Surveillance and Security Command Center

Prepared for: Presentation, viva preparation, and deep revision

Student: PATEL BINIT

Programme: B.E. COMPUTER ENGINEERING

College: GOVERNMENT ENGINEERING COLLEGE BHARUCH

University: Gujarat Technological University

Internship Context: Microsoft Elevate Program via Edunet Foundation and FICE Education

Project Duration: January 2026 to April 2026

1. How To Use This Notebook

This notebook is a complete study companion for the Border Defence AI project. It explains the system from problem statement to implementation so that the project can be presented confidently in a review, internship evaluation, or viva.

Use it in three passes:

- First read Sections 2 to 5 for the overall mental model.
- Then read Sections 6 to 12 for architecture, theory, dataset, model training, and implementation.
- Finally read Sections 13 to 18 for results, demo flow, limitations, future scope, and viva answers.

The key point to remember is simple: this is not only a YOLO detection project. It is an integrated surveillance pipeline that combines object detection, anomaly detection, spatial zone intelligence, temporal behavior analysis, alert management, dashboard visualization, and optional Azure cloud storage.

2. Project In One Page

2.1 Project Summary

Border Defence AI is a Python-based AI surveillance command center. It processes video or camera input, detects security-relevant objects using a custom YOLOv8 model, analyzes abnormal activity using machine learning and rule-based intelligence, generates priority-based alerts, stores results, and displays operational insights through a Streamlit dashboard.

2.2 Problem Solved

Traditional surveillance depends heavily on humans watching long video feeds. This creates fatigue, delayed response, missed incidents, and unstructured evidence. A practical surveillance system should automatically detect objects, understand where they are, observe their movement, decide whether the event is suspicious, and create readable alerts for operators.

This project solves that workflow at prototype level by converting raw video into structured security intelligence.

2.3 Core Input And Output

Inputs:

- Video file or camera stream.
- Custom trained YOLOv8 model weights.
- Pipeline configuration through command-line arguments.
- Optional .env configuration for Azure and email notifications.

Outputs:

- Bounding-box detections with class names and confidence scores.
- Zone violations for restricted, buffer, and observation areas.
- Temporal behavior alerts such as loitering, crowd buildup, approach trajectory, and coordinated movement.
- Anomaly scores and threat levels.
- JSON alert logs and enhanced session results.
- Annotated frames or videos when enabled.
- Dashboard visualizations.
- Optional Azure Blob Storage and Cosmos DB records.

2.4 End-To-End Flow

2.5 Main Modules

- `src/preprocessing.py`: Handles video reading, frame extraction, resizing, optical flow, and frame utilities.
 - `src/detector.py`: Loads the YOLOv8 model and returns structured detections.
 - `src/zone_analyzer.py`: Adds spatial intelligence by checking detections against surveillance zones.
 - `src/temporal_analyzer.py`: Adds behavior intelligence by analyzing objects across frames.
 - `src/anomaly.py`: Extracts features and applies Isolation Forest plus Random Forest classification.
 - `src/alert_manager.py`: Converts suspicious events into priority-based alerts.
 - `src/azure_client.py`: Handles optional Azure Blob Storage and Cosmos DB integration.
 - `src/pipeline.py`: Orchestrates the complete enhanced pipeline.
 - `dashboard/app.py`: Main Streamlit dashboard.
 - `dashboard/pages/1_..._Enhanced_Analysis.py`: Enhanced zone and temporal analysis dashboard page.
-

3. Internship And Project Context

The project was developed as part of the Microsoft Elevate internship learning journey. The internship focused on practical exposure to cloud concepts, Python programming, AI/ML, software engineering, and applied project development. The Border Defence AI system became the practical outcome of that learning.

The project theme was selected because border surveillance combines several real engineering areas:

- Computer vision for object detection.
- Machine learning for abnormal behavior detection.
- Real-time or near-real-time data processing.
- Cloud storage for evidence and logs.
- Dashboard development for operator visibility.
- Modular software architecture for maintainability.

The system is designed as an academic and internship-level prototype, but the architecture follows production-style thinking: clear modules, structured outputs, configurable pipeline, tests, logs, cloud integration, and dashboard-based monitoring.

4. Objectives

4.1 Technical Objectives

- Build a modular Python surveillance pipeline.
- Train and use a custom YOLOv8 model for seven surveillance classes.
- Process video frames sequentially.

- Detect objects and assign threat categories.
- Extract numerical features from detections and motion.
- Detect abnormal activity using machine learning.
- Add zone-based security context.
- Add temporal behavior analysis across frames.
- Generate priority-based alerts.
- Store alerts and enhanced results locally.
- Provide optional Azure cloud integration.
- Display results in a Streamlit dashboard.

4.2 Learning Objectives

- Understand YOLOv8 custom object detection.
 - Learn YOLO dataset format and preprocessing.
 - Handle dataset imbalance and CPU training limitations.
 - Understand Isolation Forest and Random Forest usage.
 - Design a modular Python application.
 - Connect backend outputs to a dashboard.
 - Learn practical Azure Blob Storage and Cosmos DB concepts.
-

5. Updated Repository Structure

The repository is organized into source code, dashboard files, datasets, trained models, scripts, tests, documentation, and generated outputs.

Project tree summary:

```
Border Surveillance Project/
  README.md
  requirements.txt
  requirements-dev.txt
  training.log
  src/
    preprocessing.py
    detector.py
    anomaly.py
    zone_analyzer.py
    temporal_analyzer.py
    alert_manager.py
    azure_client.py
    pipeline.py
  dashboard/
    app.py
    pages/
      Enhanced Analysis page
  scripts/
    check_dataset.py
    patch_v21.py
    pilot.py
    preprocess_all_datasets.py
    preprocess_balanced_v2.py
    smoke_test.py
    xview_geojson_to_yolo.py
  tests/
    test_anomaly_and_alert.py
    test_detector.py
    test_pipeline.py
    test_preprocessing.py
    test_temporal_analyzer.py
    test_zone_analyzer.py
  data/
    raw/
    processed/
    test_videos/
    alerts/
    results/
```

```
detections/  
annotated_frames/  
models/  
    border_yolo.pt  
runs/  
    detect/border_v2_balanced/  
docs/  
notebooks/
```

5.1 Root Files

- README.md is the main public-facing project documentation.
- requirements.txt contains runtime dependencies.
- requirements-dev.txt contains developer dependencies such as testing and linting tools.
- training.log stores the YOLO training output and confirms the CPU-based training environment.

5.2 Source Folder

The src folder contains the core backend logic. This is the heart of the project.

- preprocessing.py handles video loading, frame extraction, frame resizing, optical flow, and frame saving.
- detector.py wraps YOLOv8 inference and converts raw predictions into structured detection objects.
- anomaly.py extracts numerical features and applies Isolation Forest plus Random Forest threat classification.
- zone_analyzer.py checks whether detected objects are present in restricted, buffer, or observation zones.
- temporal_analyzer.py tracks objects across frames and detects movement-based suspicious behavior.
- alert_manager.py creates alerts, assigns priority, applies cooldown, writes JSON logs, and supports notifications.
- azure_client.py manages optional Azure Blob Storage and Cosmos DB integration.
- pipeline.py connects every module into one enhanced runtime workflow.

5.3 Dashboard Folder

The dashboard folder contains the Streamlit user interface.

- dashboard/app.py is the main command center dashboard.
- dashboard/pages/1_Enhanced_Analysis.py shows zone intelligence, temporal intelligence, risk trends, and detection summaries.

The dashboard reads pipeline output files. It does not directly train the model or replace the backend pipeline.

5.4 Scripts Folder

The scripts folder contains utility scripts for dataset preparation and validation.

- preprocess_balanced_v2.py is the final balanced dataset preprocessing script used for the current model workflow.
- check_dataset.py checks dataset quality, empty labels, invalid boxes, class distribution, and imbalance.
- patch_v21.py reduces vehicle dominance by capping vehicle annotations and removing some vehicle-only images.
- xview_geojson_to_yolo.py converts xView annotation format into YOLO format.
- smoke_test.py provides a quick sanity check.
- preprocess_all_datasets.py and pilot.py are older or experimental workflow scripts.

5.5 Tests Folder

The tests folder shows that the project is not only a demo but also includes engineering validation.

The tests cover:

- Detection data structures and detector behavior.
- Preprocessing and optical flow utilities.

- Anomaly detection and alert manager logic.
- Pipeline configuration and execution behavior.
- Zone analyzer logic.
- Temporal analyzer logic.

5.6 Data, Models, And Runs

- data/raw stores original dataset files.
 - data/processed stores the final YOLO-format training dataset.
 - data/test_videos stores demo videos.
 - data/alerts stores alert_log.json.
 - data/results stores enhanced_analysis.json and enhanced_session JSON files.
 - models stores custom trained model weights.
 - runs/detect/border_v2_balanced stores YOLO training artifacts, validation results, and weights.
-

6. System Architecture

The system follows a modular pipeline architecture. Each module has one focused responsibility and passes structured output to the next module.

Architecture flow:

```
Video or Camera Input
  -> Preprocessing
  -> YOLOv8 Detector
  -> Zone Analyzer
  -> Temporal Analyzer
  -> Anomaly Detector
  -> Alert Manager
```

- > Local Storage and Azure Storage
- > Streamlit Dashboard

6.1 Why Modular Architecture Was Used

A modular design makes the system easier to test, debug, extend, and explain.

Examples:

- If the detector model changes, detector.py and model weights are the main affected areas.
- If zone rules change, zone_analyzer.py can be updated independently.
- If notification channels change, alert_manager.py can be extended.
- If Azure is unavailable, the local pipeline can still run.

This separation is important because real-world AI systems must be maintainable, not just functional.

6.2 Runtime Phases

The enhanced pipeline has two conceptual phases.

Baseline phase:

- Initial frames are collected.
- Feature samples are used to establish normal behavior.
- The anomaly detector becomes ready after enough samples are collected.

Live scoring phase:

- New frames are scored for anomalies.
- Zone and temporal intelligence are combined with anomaly results.

- Alerts are generated and logged.

The anomaly system needs a minimum sample size of around 30 frames before stable scoring.

7. Data Flow In Detail

7.1 Input Source

The pipeline accepts either a video file or camera source. For presentation, a video file is safer because it produces repeatable results.

Typical command:

```
python src/pipeline.py --video data/test_videos/dota_aerial_test.mp4
```

7.2 Frame Extraction

OpenCV reads the video frame by frame. The pipeline can process every frame or skip frames depending on the frame-skip setting. Frame skipping is useful on CPU-only hardware because it reduces processing load.

7.3 Preprocessing

Each frame is prepared before detection. Preprocessing includes reading the frame, resizing or preparing it for inference, calculating optical flow when enabled, and preserving metadata such as frame index and timestamp.

Optical flow estimates motion between consecutive frames. This motion score becomes an important feature for anomaly detection.

7.4 YOLOv8 Detection

The YOLOv8 detector returns bounding boxes, class IDs, class names, and confidence values. The project uses seven target classes:

ID	Class Name
0	person
1	vehicle
2	crowd
3	military_vehicle
4	aircraft
5	ship
6	suspicious_object

Raw YOLO predictions are converted into Detection objects and collected inside a FrameResult.

7.5 Zone Analysis

Each detection is checked against configured surveillance zones. The current default logic includes restricted, buffer, and observation zones. The analyzer uses point-in-polygon logic on the object center point.

Zone analysis answers this question: where did the object appear, and is that location sensitive?

7.6 Temporal Analysis

The temporal analyzer studies behavior across frames. It uses a lightweight IoU-based tracker and a sliding window of recent frames.

It can detect:

- Sudden appearance.
- Crowd buildup.
- Loitering.
- Approach trajectory.
- Coordinated movement.

Temporal analysis answers this question: is the behavior suspicious over time?

7.7 Anomaly Detection

The anomaly module converts visual observations into numerical features. These features are scored by Isolation Forest and classified by Random Forest.

Features include detection count, class diversity, confidence values, critical object count, object location, object size, motion score, and suspicious object flag.

7.8 Alert Generation

The alert manager receives anomaly, zone, and temporal results. It creates a structured alert only when the event is important enough. It also applies cooldown so the same event does not spam repeated alerts.

Alert priorities are CRITICAL, HIGH, MEDIUM, and LOW.

7.9 Storage And Dashboard

The system writes local JSON outputs and optionally uploads evidence to Azure. The dashboard reads these generated files and displays useful charts and summaries.

Important local outputs:

- data/alerts/alert_log.json
 - data/results/enhanced_analysis.json
 - data/results/enhanced_session_*.json
 - data/annotated_frames when frame saving is enabled
-

8. Theory Foundation

8.1 Computer Vision

Computer vision is a field of artificial intelligence that allows computers to understand images and videos. In this project, computer vision is used to detect objects such as persons, vehicles, aircraft, ships, and suspicious objects from surveillance frames.

A video is treated as a sequence of images. The system processes these images one by one and extracts meaningful information from them.

8.2 Object Detection

Object detection finds and classifies objects inside an image. It is more detailed than image classification because it gives both the object category and its location.

A detection usually contains:

- Class name.
- Confidence score.
- Bounding box coordinates.
- Object size and position.

In this project, object detection is the first intelligence layer. Without detection, the system cannot know what objects are present in the scene.

8.3 Bounding Boxes

A bounding box is a rectangle around a detected object. In common image-coordinate format it can be represented as x_1, y_1, x_2, y_2 . In YOLO label format it is represented as `class_id, center_x, center_y, width, and height`, where coordinates are normalized between 0 and 1.

Bounding boxes are important because later modules use object location for zone analysis and object tracking.

8.4 YOLOv8

YOLO means You Only Look Once. It is a real-time object detection model family. YOLO predicts object locations and classes in one forward pass through the neural network.

YOLOv8 was selected because:

- It supports custom object detection training.
- It provides a clean Python API through Ultralytics.
- It supports training, validation, prediction, and export workflows.
- It is widely used and well documented.
- YOLOv8n is lightweight enough for CPU-based experimentation.

The project uses a custom YOLOv8 model trained for seven border-surveillance classes.

8.5 Confidence Score

The confidence score tells how strongly the model believes in a detection. A high confidence score means the model is more certain.

A confidence threshold is used to remove weak detections. For example, if the confidence threshold is 0.25, detections below 0.25 are ignored.

8.6 Intersection Over Union

Intersection over Union, or IoU, measures overlap between two bounding boxes.

IoU equals area of intersection divided by area of union.

IoU is used for:

- Object detection evaluation.
- Non-Max Suppression.
- Matching objects across frames in simple tracking.

A higher IoU means two boxes overlap more strongly.

8.7 Non-Max Suppression

Object detectors can sometimes predict multiple boxes around the same object. Non-Max Suppression removes duplicate overlapping boxes and keeps the best one. This improves final detection quality.

8.8 Precision And Recall

Precision tells how many predicted detections were correct.

Recall tells how many actual objects were detected.

In surveillance, both are important:

- Low precision causes false alarms.
- Low recall causes missed threats.

8.9 mAP50 And mAP50-95

Mean Average Precision, or mAP, is a standard object detection metric.

- mAP50 measures average precision at IoU threshold 0.50.
- mAP50-95 averages performance across stricter IoU thresholds from 0.50 to 0.95.

mAP50-95 is harder because it rewards accurate localization, not just rough object detection.

8.10 Optical Flow

Optical flow estimates motion between consecutive frames. It tells how pixels move from one frame to the next.

The project uses optical flow to calculate a motion score. This motion score becomes part of anomaly detection. High motion can indicate sudden activity or unusual movement.

8.11 Anomaly Detection

Anomaly detection identifies unusual patterns. In surveillance, an anomaly can be a sudden crowd, a suspicious object, unusual movement, or activity in a sensitive area.

The project uses anomaly detection because object detection alone does not fully explain whether a situation is normal or dangerous.

8.12 Isolation Forest

Isolation Forest is an unsupervised anomaly detection algorithm. It works on the idea that unusual points are easier to isolate than normal points.

It is useful here because:

- It does not require manually labeled anomaly data.
- It works on numerical feature vectors.
- It is lightweight enough for CPU-based experimentation.
- It can detect unusual combinations of detections and motion.

8.13 Random Forest Threat Classifier

Random Forest is a supervised machine learning algorithm made of many decision trees. Each tree gives a prediction and the final output is based on voting.

In this project, Random Forest is used as an extra threat classification layer. It helps convert frame-level features into a more understandable threat label.

8.14 Zone-Based Intelligence

The location of an object changes the meaning of the detection.

Example:

- A vehicle in an observation zone may be low risk.
- A vehicle in a restricted zone may require attention.
- A suspicious object in a restricted zone can become critical.

Zone intelligence adds spatial context to detection.

8.15 Temporal Intelligence

Single-frame detection cannot detect behavior such as loitering or approach direction. Temporal intelligence analyzes multiple frames and looks for behavior patterns.

The temporal analyzer helps detect:

- Objects appearing suddenly.
- Objects staying too long.
- Crowds increasing over time.
- Objects moving toward protected zones.
- Multiple objects moving together.

8.16 Alert Prioritization

Not every detection should become a critical alert. Alert priority depends on object class, location, behavior, anomaly score, and motion.

A good alert system reduces noise while still highlighting important events.

9. Dataset And Model Training

9.1 Target Classes

The final custom model uses seven classes:

ID	Class	Meaning
0	person	Individual human presence
1	vehicle	General vehicles
2	crowd	Group activity or dense people
3	military_vehicle	Military-style vehicle
4	aircraft	Plane or aerial vehicle
5	ship	Marine vessel
6	suspicious_object	Potentially risky object

9.2 Dataset Sources

The final balanced v2 preprocessing workflow uses:

- xView for satellite-style objects such as vehicles, ships, aircraft, and other aerial objects.
- VisDrone for persons, vehicles, and crowd-like drone imagery.
- DOTA for aerial objects such as aircraft, ships, and vehicles.

VEDAI was explored during the project, but the final balanced v2 preprocessing script removes VEDAI from the active workflow to keep the dataset more manageable and focused.

9.3 Why Multiple Datasets Were Combined

A border-surveillance dataset needs multiple object types and viewing angles. No single public dataset fully matched the project needs, so multiple datasets were combined and remapped into the final seven classes.

This made the model more suitable for mixed surveillance scenarios.

9.4 YOLO Dataset Format

The processed dataset follows standard YOLO structure:

```
images/train
images/val
images/test
labels/train
labels/val
labels/test
data.yaml
```

Each label file contains one line per object:

```
class_id center_x center_y width height
```

The coordinate values are normalized between 0 and 1.

9.5 Dataset Imbalance

The dataset was heavily imbalanced. Vehicle annotations were dominant, while important classes such as `military_vehicle` and `suspicious_object` had fewer samples.

This matters because imbalance can cause:

- Over-detection of majority classes.
- Poor recall on rare classes.
- Misleading validation metrics.
- Weak performance on critical security classes.

9.6 Balanced v2 Strategy

The final script `preprocess_balanced_v2.py` applies balancing logic suitable for CPU training.

Important steps:

- Reduce excessive vehicle-only samples.
- Preserve rare-class images.
- Use letterbox resizing to 640 by 640.
- Adjust labels correctly after resizing.
- Split data into train, validation, and test sets.
- Keep the dataset smaller and more realistic for CPU training.

Approximate balancing ideas used in the script:

- Keep a limited ratio of vehicle-only images.
- Keep more images containing rare classes.
- Protect rare classes from aggressive removal.
- Use an 80/10/10 train-validation-test split.

9.7 Hardware Constraint

Training was performed on a CPU-only laptop setup with Intel i5 processor, 16 GB DDR5 RAM, and 512 GB SSD. There was no dedicated GPU for model training.

This affected training in practical ways:

- Batch size had to stay very small.
- Batch size greater than 2 caused crashes in previous trials.
- Training took a long time.
- YOLOv8n was selected because it is lightweight.
- Dataset size and balance had to be controlled carefully.

This is an important honest engineering point for presentation.

9.8 Training Configuration

Final model training summary:

Item	Value
Model	YOLOv8n
Dataset	Balanced v2
Datasets Used	DOTA, xView, VisDrone
Image Size	640
Epochs	30
Batch Size	2
Workers	0
Device	CPU
Training Run	runs/detect/border_v2_balanced

9.9 Final Model Metrics

Custom YOLOv8 Model Performance, v2 Balanced, 30 epochs:

Class	Images	Instances	Precision	Recall	mAP50	mAP50-95
all	1685	30195	0.701	0.457	0.478	0.285
person	603	10387	0.463	0.283	0.293	0.096
vehicle	999	8170	0.327	0.398	0.270	0.155
crowd	24	149	0.713	0.367	0.483	0.197
military_vehicle	4	8	1.000	0.000	0.000	0.000
aircraft	425	2353	0.831	0.622	0.668	0.479
ship	527	8169	0.780	0.821	0.870	0.561
suspicious_object	116	959	0.791	0.706	0.763	0.505

9.10 Metric Interpretation

The strongest classes are ship, suspicious_object, and aircraft. These classes have comparatively better mAP and recall.

The weaker classes are military_vehicle, person, and vehicle. The military_vehicle class is especially weak because there are very few examples in validation. This is a dataset limitation, not simply a code problem.

Overall mAP50 of 0.478 shows that the model learned useful detection behavior, but there is still room for improvement through better data, longer training, and GPU support.

9.11 Inference Speed

The custom model achieved roughly:

- Preprocessing around 1.4 ms per image.
- CPU inference around 114.7 ms per image.
- Postprocessing around 1.4 ms per image.

This is suitable for demonstration and near-real-time experimentation, but production real-time deployment would benefit from GPU or edge accelerator hardware.

10. Implementation Walkthrough

10.1 preprocessing.py

The preprocessing module prepares the video data before AI analysis.

Responsibilities:

- Open video files or camera sources.
- Read frames sequentially using OpenCV.
- Resize frames when needed.
- Compute optical flow for motion analysis.
- Save frames when requested.
- Provide video metadata.

Why it matters:

- YOLO works on images, not raw video files.
- Motion information improves anomaly detection.
- Keeping preprocessing separate makes the pipeline cleaner.

10.2 detector.py

The detector module wraps the YOLOv8 model.

Responsibilities:

- Load the YOLO model once.
- Run inference on each frame.
- Filter predictions by confidence and IoU thresholds.
- Convert raw predictions into structured Detection objects.
- Return a FrameResult for every processed frame.
- Annotate frames with boxes and labels.

Important structures:

- Detection stores class, confidence, box, normalized position, and threat metadata.
- FrameResult stores frame index, timestamp, detections, and summary features.
- BorderDetector is the main detector class.

Threat mapping used by the detector gives an initial security meaning to each class:

- person: medium.
- vehicle: low.
- crowd: high.
- military_vehicle: critical.
- aircraft: high.
- ship: medium.
- suspicious_object: critical.

This mapping is not the final alert decision. It is one signal used later by anomaly, zone, and alert modules.

10.3 zone_analyzer.py

The zone analyzer adds spatial context.

Default zones:

- Border or restricted zone.
- Buffer zone.
- Observation zone.

The analyzer checks if a detection center point lies inside a polygon zone. If a high-risk object is found in a sensitive zone, it creates a zone violation.

Zone priority concept:

- Restricted zone generally produces critical priority.
- Buffer zone generally produces high priority.
- Observation zone generally produces medium priority.
- Safe area generally produces low priority.

This module is important because a detection becomes more meaningful when the system knows where it happened.

10.4 temporal_analyzer.py

The temporal analyzer studies behavior across time.

Responsibilities:

- Maintain a sliding window of recent frames.
- Track objects using bounding-box IoU matching.
- Detect sudden appearance.
- Detect crowd buildup.
- Detect loitering.
- Detect approach trajectory.
- Detect coordinated movement.
- Produce temporal risk scores and temporal alerts.

Important thresholds in the implementation include:

- IoU tracking threshold around 0.3.
- Sliding window default around 30 frames.
- Loitering threshold around 10 frames.
- Sudden appearance and crowd buildup thresholds for behavior detection.

Why it matters:

A single frame can tell what is present. Temporal analysis tells what is happening.

10.5 anomaly.py

The anomaly module is the machine learning intelligence layer.

Main responsibilities:

- Convert frame-level observations into numerical features.
- Train or fit an Isolation Forest baseline.
- Score live frames for abnormal behavior.
- Use Random Forest classification for threat labeling.
- Produce an AnomalyResult object.

Feature vector examples:

- Detection count.
- Class diversity.
- Average confidence.
- Maximum confidence.
- Critical object count.
- Average center x and center y.
- Average width.
- Motion score.
- Suspicious object flag.

The current implementation uses a dual ML approach:

- Isolation Forest for unsupervised anomaly scoring.
- Random Forest for supervised-style threat classification using generated labels from anomaly behavior.

This gives the system both anomaly scoring and a more readable threat label.

10.6 alert_manager.py

The alert manager turns intelligence output into actionable alerts.

Responsibilities:

- Create alert objects.
- Assign priority.
- Apply cooldown.
- Write alerts into a rolling JSON log.
- Trigger email notification when configured.
- Save alert records through Azure integration when enabled.

Alert priorities:

- CRITICAL.
- HIGH.
- MEDIUM.
- LOW.

Cooldown is important because the same object can remain visible for many frames. Without cooldown, the system would generate too many duplicate alerts.

10.7 azure_client.py

The Azure client handles optional cloud storage.

Supported cloud actions:

- Upload alert frames to Azure Blob Storage.
- Upload session result JSON files.
- Upload annotated frames.
- Save alert metadata to Azure Cosmos DB.
- Query recent alerts.
- Run health checks.
- Get alert statistics.

The implementation uses lazy imports and graceful fallback. This means local development can continue even when Azure credentials are not configured.

10.8 pipeline.py

The pipeline is the main orchestration layer.

Important classes:

- EnhancedConfig stores CLI and runtime configuration.
- EnhancedSession stores session-level counters and results.
- EnhancedPipeline connects all modules and processes frames.

For every frame, the enhanced pipeline performs:

1. Preprocess the frame.

2. Run YOLO detection.
3. Save annotated frame or video if enabled.
4. Run zone analysis if enabled.
5. Run temporal analysis if enabled.
6. Extract anomaly features.
7. Collect baseline samples if anomaly model is not ready.
8. Score frame during live phase.
9. Merge anomaly, zone, and temporal reasons.
10. Generate alerts through alert_manager.py.
11. Upload important artifacts through azure_client.py if configured.
12. Save enhanced analysis data for dashboard.

10.9 Enhanced Configuration Options

Important command-line options include:

- --video for input video path.
- --camera for webcam or camera index.
- --model for custom model path.
- --confidence for detection threshold.
- --iou for IoU threshold.
- --frame-skip for reducing CPU load.
- --no-flow to disable optical flow.
- --no-zones to disable zone analysis.
- --no-temporal to disable temporal analysis.
- --save-frames to save annotated frames.
- --save-video to save annotated video.
- --max-frames to limit processing for fast demos.
- --verbose for detailed logs.

10.10 Dashboard Implementation

The dashboard is built with Streamlit. It reads local output files generated by the pipeline.

Main dashboard features:

- Alert KPIs.
- Alert priority distribution.
- Recent alert list.
- Charts for alert activity.
- Manual notification controls.
- Auto-refresh behavior.
- Security command center styling.

Enhanced analysis page features:

- Zone violation analysis.
- Temporal alert breakdown.
- Risk score timeline.
- Detection trends.
- Tracking and behavior summary.

This separation between backend pipeline and frontend dashboard is important because the dashboard can review results without directly running model inference.

10.11 Tests

The repository includes tests for major modules.

Test coverage areas:

- Detector object structures and model wrapper behavior.
- Preprocessing and optical flow.
- Anomaly detector and alert manager.
- Pipeline configuration and execution.
- Zone analyzer.
- Temporal analyzer.

Testing improves confidence that the modules work independently and together.

11. Configuration And Environment

11.1 Environment Variables

The project uses environment variables for sensitive configuration. Actual secret values should never be committed or shared.

Common variables include:

- AZURE_STORAGE_CONNECTION_STRING.
- AZURE_STORAGE_CONTAINER_ALERTS.
- AZURE_STORAGE_CONTAINER_RESULTS.
- AZURE_COSMOS_ENDPOINT.
- AZURE_COSMOS_KEY.
- AZURE_COSMOS_DATABASE.
- AZURE_COSMOS_CONTAINER.

- SENDGRID_API_KEY.
- ALERT_FROM_EMAIL.
- ALERT_TO_EMAIL.
- SMTP_HOST.
- SMTP_PORT.
- SMTP_USERNAME.
- SMTP_PASSWORD.

11.2 Why Environment Variables Are Used

Environment variables separate code from deployment-specific configuration.

Benefits:

- Safer secret handling.
- Same code can run locally or in cloud.
- Configuration can change without source-code edits.
- Public repository can avoid exposing private credentials.

12. How To Run The Project

12.1 Create And Activate Environment

Linux or WSL:

```
python -m venv venv
```

```
source venv/bin/activate
```

Windows PowerShell:

```
python -m venv venv  
.\venv\Scripts\Activate.ps1
```

12.2 Install Dependencies

```
pip install -r requirements.txt
```

12.3 Check Dataset Quality

```
python scripts/check_dataset.py
```

This checks image count, label count, empty labels, invalid boxes, corrupt images, and class balance.

12.4 Run Enhanced Pipeline

```
python src/pipeline.py --video data/test_videos/dota_aerial_test.mp4 --save-frames --save-video
```

12.5 Run A Fast Demo

```
python src/pipeline.py --video data/test_videos/dota_aerial_test.mp4 --max-frames 50 --save-frames
```

This is useful for presentation because it reduces waiting time.

12.6 Run Dashboard

```
streamlit run dashboard/app.py
```

12.7 Run Tests

```
pytest tests/
```

13. Results And Observations

13.1 Model Results

The final custom YOLOv8 model achieved:

- Overall mAP50: 0.478.
- Overall mAP50-95: 0.285.
- Precision: 0.701.
- Recall: 0.457.

The model is strongest for ship, suspicious_object, and aircraft. It is weaker for military_vehicle because that class has very few validation examples. Person and vehicle detection can also improve with better data and more training.

13.2 Runtime Session Example

One enhanced pipeline session on dota_aerial_test.mp4 produced approximately:

- Total frames: 195.
- Frames scored: 195.

- Total detections: 2079.
- Alerts raised: 133.
- Zone violations: 2079.
- Temporal alerts: 25.
- Average inference time: about 173.81 ms.
- Effective FPS: about 0.73.

This shows that the system is doing more than detection. It is combining detection with zone analysis, temporal behavior analysis, anomaly scoring, and alert generation.

13.3 Alert Output

Alerts are stored in data/alerts/alert_log.json. A typical alert contains:

- Timestamp.
- Priority.
- Frame index.
- Detection count.
- Motion score.
- Reasons.
- Notification status.

Example alert interpretation:

A CRITICAL alert may be generated because a vehicle is detected in an observation or sensitive zone and temporal logic also detects coordinated movement or loitering behavior.

13.4 Dashboard Output

The dashboard helps the user understand:

- How many alerts were generated.
- Which priorities are most common.
- Which zones have more violations.
- Which temporal behaviors occurred.
- How detections changed over time.
- Which alerts need operator attention.

13.5 Result Interpretation

The project successfully demonstrates an integrated AI surveillance workflow. It does not only detect objects; it also reasons about object type, location, movement behavior, anomaly score, alert priority, storage, and visualization.

This integration is the main achievement of the project.

14. Azure Integration

14.1 Purpose

Azure integration makes the project closer to a real surveillance command center. Instead of keeping every output only on a local machine, important artifacts can be stored in the cloud.

14.2 Azure Blob Storage

Azure Blob Storage can store:

- Alert frames.
- Annotated frames.
- Session result JSON files.
- Dashboard-compatible output artifacts.

14.3 Azure Cosmos DB

Azure Cosmos DB can store structured alert metadata such as:

- Alert ID.
- Timestamp.
- Priority.
- Reasons.
- Detection count.
- Source video.
- Frame index.

14.4 Graceful Fallback

The Azure client is designed so that the local system can continue working even when Azure credentials are not available. This is a good engineering practice because cloud configuration should not break local testing.

15. Alert System

15.1 Alert Sources

Alerts may come from:

- Critical object detections.
- Zone violations.
- Temporal behavior alerts.
- Anomaly scores.
- High motion activity.
- Combined enhanced reasons.

15.2 Alert Priority Levels

- CRITICAL means immediate attention is required.
- HIGH means serious suspicious activity.
- MEDIUM means the event should be reviewed.
- LOW means informational or weak suspicious signal.

15.3 Automatic Alerts

Automatic alerts are generated during pipeline execution. They are saved into JSON logs and can trigger email or cloud actions when configured.

15.4 Manual Notifications

The dashboard supports manual email notification actions. This gives the operator control when a specific event needs to be sent or escalated.

15.5 Cooldown

Cooldown prevents repeated alerts for the same continuous event. Without cooldown, the system could generate hundreds of duplicate alerts while the same object remains visible.

16. Presentation Demo Flow

16.1 Start With The Problem

Explain that continuous surveillance is difficult for humans because it causes fatigue and delayed response. AI can assist by detecting objects and converting video into structured alerts.

16.2 Show The Architecture

Use this sentence:

The system follows the flow: video input, preprocessing, YOLOv8 detection, zone analysis, temporal analysis, anomaly detection, alert generation, storage, and dashboard visualization.

16.3 Explain The Model

Mention:

- Custom YOLOv8n model.
- Seven target classes.
- Balanced v2 dataset.
- CPU-only training.
- Final mAP50 of 0.478.
- Stronger classes: ship, aircraft, suspicious_object.
- Weaker class: military_vehicle due to low data.

16.4 Run The Pipeline

Use a short demo command:

```
python src/pipeline.py --video data/test_videos/dota_aerial_test.mp4 --max-frames 50 --save-frames
```

16.5 Show Alerts

Open or explain:

```
data/alerts/alert_log.json
```

Explain that alerts contain timestamp, priority, reason, frame index, detection count, and notification status.

16.6 Show Dashboard

Run:

```
streamlit run dashboard/app.py
```

Show:

- Alert count.
- Priority distribution.
- Recent alerts.
- Enhanced analysis page.
- Zone and temporal charts.

16.7 Close With Limitations And Future Scope

End honestly by explaining that the current system is a strong prototype, but real deployment would need live camera integration, better hardware, more balanced data, and production-level infrastructure.

17. Viva Questions And Answers

Q1. What is your project about?

The project is an AI-based border surveillance system. It processes video frames, detects security-related objects using YOLOv8, analyzes anomalies, checks zone violations, studies temporal behavior, generates alerts, stores results, and displays insights on a Streamlit dashboard.

Q2. Why did you choose YOLOv8?

YOLOv8 is modern, fast, supports custom training, and provides a complete workflow for training, validation, inference, and export. The YOLOv8n version is lightweight enough for CPU-based experimentation.

Q3. What are the seven classes?

The seven classes are person, vehicle, crowd, military_vehicle, aircraft, ship, and suspicious_object.

Q4. What is the difference between object detection and anomaly detection?

Object detection identifies what object is present and where it is located. Anomaly detection checks whether the overall situation is unusual based on features such as object count, motion, class diversity, suspicious class presence, and location context.

Q5. Why did you use Isolation Forest?

Isolation Forest can detect anomalies without needing manually labeled anomaly data. It works well with numerical features and is lightweight enough for CPU-based experimentation.

Q6. What is the role of Random Forest?

Random Forest acts as an additional threat classification layer. It helps convert extracted features into more interpretable threat categories.

Q7. What is zone analysis?

Zone analysis checks whether detected objects are inside restricted, buffer, or observation zones. It adds spatial context to the detection result.

Q8. Why is zone analysis important?

The same object can have different risk depending on its location. A vehicle in a normal area may be low risk, but a vehicle in a restricted area may require attention.

Q9. What is temporal analysis?

Temporal analysis studies behavior across multiple frames. It detects patterns such as sudden appearance, crowd buildup, loitering, approach trajectory, and coordinated movement.

Q10. Why is temporal analysis useful?

Some suspicious behaviors cannot be understood from one frame. Loitering, approach direction, and crowd buildup require observation over time.

Q11. What is optical flow?

Optical flow estimates motion between consecutive frames. In this project, it contributes a motion score that helps anomaly detection.

Q12. What is mAP50?

mAP50 is mean average precision at an IoU threshold of 0.50. It is a standard object detection metric.

Q13. What is precision?

Precision measures how many predicted detections were correct. High precision means fewer false positives.

Q14. What is recall?

Recall measures how many actual objects were detected. High recall means fewer missed objects.

Q15. What were your final model metrics?

The final balanced v2 YOLOv8 model achieved approximately 0.478 mAP50, 0.285 mAP50-95, 0.701 precision, and 0.457 recall overall.

Q16. Which classes performed best?

Ship, suspicious_object, and aircraft performed better compared to other classes.

Q17. Which class performed worst and why?

Military_vehicle performed worst because the validation set had very few examples. This is mainly a dataset limitation.

Q18. What hardware did you use?

Training was done on a CPU-only laptop with Intel i5 processor, 16 GB RAM, and 512 GB SSD. No dedicated GPU was available.

Q19. How did hardware affect training?

The CPU-only setup forced a small batch size, small model choice, controlled dataset size, and longer training time. Batch size greater than 2 caused crashes.

Q20. What does the alert manager do?

The alert manager creates structured alerts, assigns priority, applies cooldown, writes alerts to JSON, and optionally sends notifications or saves records to Azure.

Q21. What does the dashboard show?

The dashboard shows alert metrics, priority distribution, recent alerts, zone violations, temporal behavior insights, and detection trends.

Q22. How is Azure used?

Azure Blob Storage can store alert frames and session outputs. Azure Cosmos DB can store structured alert metadata. The local system continues working even without Azure configuration.

Q23. Is the system real time?

The architecture supports real-time or near-real-time processing, but on the current CPU-only laptop it runs slower than true real time. GPU or edge hardware would improve speed.

Q24. What was the biggest technical challenge?

The biggest challenges were dataset imbalance, CPU-only training, integrating multiple modules, and making detection, anomaly analysis, alerting, cloud support, and dashboard visualization work together.

Q25. What is the main strength of the project?

The main strength is integration. The project combines object detection, anomaly detection, zone analysis, temporal analysis, alert management, dashboard visualization, and Azure support in one modular system.

18. Important Technical Terms

YOLO: A single-stage object detector that predicts boxes and classes in one pass.

Bounding box: A rectangle around a detected object.

Confidence: The model certainty for a prediction.

IoU: Intersection over Union, used to measure overlap between boxes.

mAP: Mean Average Precision, a common object detection performance metric.

Optical flow: Motion estimation between frames.

Isolation Forest: An unsupervised anomaly detection algorithm.

Random Forest: An ensemble machine learning algorithm based on multiple decision trees.

Zone violation: A detected object appearing inside a sensitive area.

Temporal alert: An alert based on behavior across multiple frames.

Cooldown: A mechanism to prevent repeated duplicate alerts.

Azure Blob Storage: Cloud object storage for files such as images and JSON outputs.

Cosmos DB: Azure database service for structured records such as alert metadata.

19. Limitations

19.1 Dataset Limitations

- The dataset is built from public datasets, not real border-only surveillance footage.
- Some classes have limited examples.
- Military_vehicle has weak performance due to very low sample count.
- Different datasets have different camera angles, resolutions, and annotation styles.

19.2 Hardware Limitations

- Training was CPU-only.
- Batch size had to remain very small.
- Training took a long time.
- True real-time deployment would need GPU or edge accelerator hardware.

19.3 Model Limitations

- Detection accuracy depends on dataset quality.
- Small objects can be difficult to detect.
- Fog, low light, blur, and weather can reduce accuracy.
- False positives and false negatives are still possible.

19.4 System Limitations

- Live CCTV or RTSP deployment is future scope.
 - Dashboard reads generated outputs instead of controlling a deployed multi-camera network.
 - Azure features require valid credentials and cloud configuration.
-

20. Future Scope

Possible improvements:

- Real-time RTSP camera stream integration.
- Edge deployment using NVIDIA Jetson or similar hardware.

- Larger custom border-surveillance dataset.
 - Better tracking using DeepSORT, ByteTrack, or improved optical-flow tracking.
 - More accurate anomaly labels and supervised behavior classification.
 - GIS or map-based visualization.
 - SMS, WhatsApp, or mobile push notifications.
 - Cloud-hosted dashboard deployment.
 - Role-based dashboard access.
 - Automated PDF report generation for alerts.
 - Model optimization using ONNX, TensorRT, or OpenVINO.
-

21. What To Emphasize During Presentation

Emphasize these points:

- The project is an integrated surveillance command center, not only a detection model.
- YOLOv8 detects objects, but zone and temporal modules add context.
- Anomaly detection converts detection features into suspicious activity scores.
- Alert manager converts intelligence into actionable security events.
- Dashboard makes outputs understandable for operators.
- Azure integration makes the system cloud-ready.
- Dataset imbalance and CPU limitations were handled honestly and practically.

Strong explanation line:

My project converts raw surveillance video into structured security intelligence by combining object detection, anomaly detection, spatial zone analysis, temporal behavior analysis, alerting, and dashboard visualization.

22. Final Revision Checklist

Before presentation, revise these points:

- Seven classes and why they were selected.
- YOLOv8 and why it was chosen.
- Difference between detection, anomaly, zone, and temporal analysis.
- Dataset sources and balancing strategy.
- Final model metrics and what they mean.
- Why military_vehicle performed poorly.
- How alert priority is decided.
- What dashboard shows.
- What Azure stores.
- Limitations and future scope.
- Demo command and dashboard command.

23. Final Summary

Border Defence AI is a modular AI surveillance command center developed as an internship project. It processes video input, detects border-security-related objects using a custom YOLOv8 model, analyzes object location through zones, studies movement over time through temporal analysis, detects anomalies using machine learning, generates priority-based alerts, stores results locally or in Azure, and presents insights through a Streamlit dashboard.

The project demonstrates practical engineering across computer vision, machine learning, Python backend design, cloud integration, dashboard development, dataset preprocessing, model training, and

testing. The most important achievement is the successful integration of multiple intelligent modules into one working surveillance pipeline.